Assignment 1

Instructions

- Unless specified otherwise,
  - you can use anything in the Python Standard Library;
  - don't use any third-party library including NumPy in your code for this assignment;
  - you can assume all user inputs are always valid and require no validation.
- The blue texts in the Sample Run(s) section of each question refers to the user inputs.
- Please follow closely the format of the sample output for each question. Your program should produce **exactly** the same output as the sample (same text, symbols, letter case, spacing, etc.). The output for each question should end with a single newline character, which has been provided by the `print()` function by default.
- Please follow exactly the specified name, parameter list and return value(s) for the functions to be written if a question has stated (to facilitate our grading). You may define additional (inner) functions for specific tasks or further task decomposition in any question if you see fit.
- **For questions that require you to define specific functions, your main client code (for calling and testing the functions) must be put under an `if __name__ == '__main__'`: check because we may import your script into our test scripts calling your functions in our own ways for grading.**
- **Name your script files as `q1.py` for Question 1, `q2.py` for Question 2, …, `q5.py` for Question 5 (case-sensitive). Using other names affects our marking and may result in deduction.**
- Your source files will be tested in script mode rather than interactive mode.
- You are highly encouraged to annotate your functions to provide type information for parameters and return values, and to include suitable comments in your code to improve the readability.
- You should follow the standard PEP8 style guide for writing Python code. Install the `pycodestyle` package via `pip install pycodestyle`. Run the command `pycodestyle <your_script>.py` to check for any PEP8 style violations, which may lead to mark deductions. You should fix all the detected issues before submission.

Question 1 (20%)

Imagine that some characters from the comedy *"Zootopia 2"* come to CUHK for an exchange program! Their academic progress and performance are recorded in four separate Python lists as shown below. Each element in the lists includes the student ID, full name, number of completed credits, cumulative GPA, and the timestamp of the last access.

```
nick = [1155123456, "Nick Wilde", 28, 3.86, "19/08/2025 08:03:54 AM"]
judy = [1155987543, "Judy Hopps", 37, 2.95, "07/12/2025 10:08:33 PM"]
gary = [1155654321, "Gary De'Snake", 42, 3.56, "23/01/2026 11:40:01 PM"]
flash = [1155998877, "Flash Slothmore", 24, 3.12, "01/01/2026 09:06:36 AM"]
```

Write a script `q1.py` to print the following data table based on these lists using the `str.format()` method or *f-string* syntax.

Sample Run

```
         1         2         3         4         5         6         7
12345678901234567890123456789012345678901234567890123456789012345678901234567890123456789012345678
Student ID | Full Name       | Credits | GPA  | Progress % | Last Access
1155123456 | Nick Wilde      | 28      | 3.86 |     53.85% | 08/19/25 08:03:54
1155987543 | Judy Hopps      | 37      | 2.95 |     71.15% | 12/07/25 22:08:33
1155654321 | Gary De'Snake   | 42      | 3.56 |     80.77% | 01/23/26 23:40:01
1155998877 | Flash Slothmore | 24      | 3.12 |     46.15% | 01/01/26 09:06:36
```

Notes:

- The top two lines (in blue color) serve as a ruler that helps you visualize the column widths and spacing requirements more clearly. Your program should skip printing them.
- The table uses a pipe symbol (|) with a single space before and after it to separate columns.
- Refer to the table below for the width (# characters) for each column in the data table.

Field	**Width**
Student ID	10
Full Name	15
Credits	7
GPA	4
Progress %	10
Last Access	17

- The Credits field means the number of credits the student has taken thus far. Each of the students must finish a total of 52 credits to complete the study.
- The Progress % field is then derived from dividing Credits by 52.
- There should be no trailing spaces at the end of each row of the printed table.

Hints:

The `datetime.strptime(date_string, format)` method can ease the task of formatting the Last Access field. See the following example.

```
from datetime import datetime as dt

today = "20/1/2026"  # A date string

# Parse the date string into a datetime object
datetime_obj = dt.strptime(today, "%d/%m/%Y")

# Print the datetime object in another specified format
print(f'{datetime_obj:%y/%m/%d}')  # Prints 26/01/20
```

Refer to the Format Codes documentation to know which format specifiers should be used to output the date-time format required in the data table.

Question 2 (20%)

Write a script `q2.py` that defines a function `pascal_triangle(n: int) -> None`. The function accepts a non-negative integer *n* and prints a *Pascal's triangle*, which consists of *n* + 1 rows (from 0th row to *n*th row).

The entry in the *n*th row and *k*th column of Pascal's triangle is given by:

$$\binom{n}{k} = \binom{n-1}{k-1} + \binom{n-1}{k} \quad \text{or} \quad \binom{n}{k} = \frac{n!}{k!\,(n-k)!}$$

for any non-negative integer *n* and any integer 0 ≤ *k* ≤ *n*.

These two formulae can be depicted as follows:

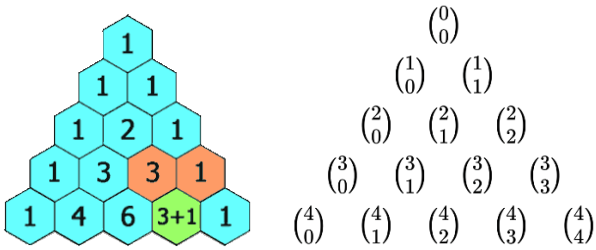

Image source: Wikipedia

Note some output formatting requirements here. Suppose that the maximum number in the triangle is *m* and *m* has *d* digits. Each entry in the triangle is center-aligned (hint: use *f-string*) in a fixed width of *d* characters if *d* is odd, or *d* + 1 characters if *d* is even.

For example, the maximum entry in the following triangle is 10, which has 2 digits (even). We will format every entry as 2 + 1 = 3 characters long with space(s) added. The '-' symbol in the following output denotes a space in each entry for clarity. For single-digit numbers, they have one '-' before and after the number. The '␣' symbol denotes a single space character, which gets printed before the leftmost entry on a row and as separators between entries.

```
Enter num: 5↵
␣␣␣␣␣␣␣␣␣-1-
␣␣␣␣␣␣␣-1-␣-1-
␣␣␣␣␣-1-␣-2-␣-1-
␣␣␣-1-␣-3-␣-3-␣-1-
␣-1-␣-4-␣-6-␣-4-␣-1-
-1-␣-5-␣10-␣10-␣-5-␣-1-
```

As another example, the maximum entry in the following triangle is 252, which has 3 digits. So, we format every entry as 3 characters long.

```
Enter num: 10↵
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣-1-
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣-1-␣-1-
␣␣␣␣␣␣␣␣␣␣␣␣␣␣-1-␣-2-␣-1-
␣␣␣␣␣␣␣␣␣␣␣␣-1-␣-3-␣-3-␣-1-
␣␣␣␣␣␣␣␣␣␣-1-␣-4-␣-6-␣-4-␣-1-
␣␣␣␣␣␣␣␣␣-1-␣-5-␣10-␣10-␣-5-␣-1-
␣␣␣␣␣␣␣-1-␣-6-␣15-␣20-␣15-␣-6-␣-1-
␣␣␣␣␣-1-␣-7-␣21-␣35-␣35-␣21-␣-7-␣-1-
␣␣␣-1-␣-8-␣28-␣56-␣70-␣56-␣28-␣-8-␣-1-
␣-1-␣-9-␣36-␣84-␣126␣126␣84-␣36-␣-9-␣-1-
-1-␣10-␣45-␣120␣210␣252␣210␣120␣45-␣10-␣-1-
```

In the following sample runs, we represent all space characters using the '␣' symbol only. For proper display, let's assume n is always ≤ 16 in your testing.

Sample Runs

```
Enter num: 0↵
1
```

```
Enter num: 1↵
␣1
1␣1
```

```
Enter num: 2↵
␣␣1
␣1␣1
1␣2␣1
```

```
Enter num: 3↵
␣␣␣1
␣␣1␣1
␣1␣2␣1
1␣3␣3␣1
```

```
Enter num: 4↵
␣␣␣␣1
␣␣␣1␣1
␣␣1␣2␣1
␣1␣3␣3␣1
1␣4␣6␣4␣1
```

```
Enter num: 5↵
␣␣␣␣␣␣␣␣␣␣1␣
␣␣␣␣␣␣␣␣␣1␣␣␣1␣
␣␣␣␣␣␣␣1␣␣␣2␣␣␣1␣
␣␣␣␣␣1␣␣␣3␣␣␣3␣␣␣1␣
␣␣␣1␣␣␣4␣␣␣6␣␣␣4␣␣␣1␣
␣1␣␣␣5␣␣10␣␣10␣␣␣5␣␣␣1␣
```

```
Enter num: 10↵
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣1␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣2␣␣␣1␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣3␣␣␣3␣␣␣1␣
␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣4␣␣␣6␣␣␣4␣␣␣1␣
␣␣␣␣␣␣␣␣␣␣1␣␣␣5␣␣10␣␣10␣␣␣5␣␣␣1␣
␣␣␣␣␣␣␣␣1␣␣␣6␣␣15␣␣20␣␣15␣␣␣6␣␣␣1␣
␣␣␣␣␣␣1␣␣␣7␣␣21␣␣35␣␣35␣␣21␣␣␣7␣␣␣1␣
␣␣␣␣1␣␣␣8␣␣28␣␣56␣␣70␣␣56␣␣28␣␣␣8␣␣␣1␣
␣␣␣1␣␣␣9␣␣36␣␣84␣␣126␣126␣84␣␣36␣␣␣9␣␣␣1␣
␣1␣␣10␣␣45␣␣120␣210␣252␣210␣120␣45␣␣10␣␣␣1␣
```

```
Enter num: 13↵
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣2␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣3␣␣␣␣␣3␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣4␣␣␣␣␣6␣␣␣␣␣4␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣5␣␣␣␣10␣␣␣␣10␣␣␣␣␣5␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣6␣␣␣␣15␣␣␣␣20␣␣␣␣15␣␣␣␣␣6␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣7␣␣␣␣21␣␣␣␣35␣␣␣␣35␣␣␣␣21␣␣␣␣␣7␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣8␣␣␣␣28␣␣␣␣56␣␣␣␣70␣␣␣␣56␣␣␣␣28␣␣␣␣␣8␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣9␣␣␣␣36␣␣␣␣84␣␣␣␣126␣␣␣126␣␣␣84␣␣␣␣36␣␣␣␣␣9␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣1␣␣␣␣10␣␣␣␣45␣␣␣␣120␣␣␣210␣␣␣252␣␣␣210␣␣␣120␣␣␣45␣␣␣␣10␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣1␣␣␣␣11␣␣␣␣55␣␣␣␣165␣␣␣330␣␣␣462␣␣␣462␣␣␣330␣␣␣165␣␣␣55␣␣␣␣11␣␣␣␣␣1␣␣
␣␣␣␣␣1␣␣␣␣12␣␣␣␣66␣␣␣␣220␣␣␣495␣␣␣792␣␣␣924␣␣␣792␣␣␣495␣␣␣220␣␣␣66␣␣␣␣12␣␣␣␣␣1␣␣
␣␣1␣␣␣␣13␣␣␣␣78␣␣␣␣286␣␣␣715␣␣1287␣␣1716␣␣1716␣␣1287␣␣␣715␣␣␣286␣␣␣78␣␣␣␣13␣␣␣␣␣1␣␣
```

```
Enter num: 16↵
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣2␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣3␣␣␣␣␣3␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣4␣␣␣␣␣6␣␣␣␣␣4␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣5␣␣␣␣␣10␣␣␣␣␣10␣␣␣␣␣5␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣6␣␣␣␣␣15␣␣␣␣␣20␣␣␣␣␣15␣␣␣␣␣6␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣7␣␣␣␣␣21␣␣␣␣␣35␣␣␣␣␣35␣␣␣␣␣21␣␣␣␣␣7␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣8␣␣␣␣␣28␣␣␣␣␣56␣␣␣␣␣70␣␣␣␣␣56␣␣␣␣␣28␣␣␣␣␣8␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣9␣␣␣␣␣36␣␣␣␣␣84␣␣␣␣␣126␣␣␣␣126␣␣␣␣84␣␣␣␣␣36␣␣␣␣␣9␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣10␣␣␣␣␣45␣␣␣␣␣120␣␣␣␣210␣␣␣␣252␣␣␣␣210␣␣␣␣120␣␣␣␣45␣␣␣␣␣10␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣11␣␣␣␣␣55␣␣␣␣␣165␣␣␣␣330␣␣␣␣462␣␣␣␣462␣␣␣␣330␣␣␣␣165␣␣␣␣55␣␣␣␣␣11␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣12␣␣␣␣␣66␣␣␣␣␣220␣␣␣␣495␣␣␣␣792␣␣␣␣924␣␣␣␣792␣␣␣␣495␣␣␣␣220␣␣␣␣66␣␣␣␣␣12␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣13␣␣␣␣␣78␣␣␣␣␣286␣␣␣␣715␣␣␣␣1287␣␣␣␣1716␣␣␣␣1716␣␣␣␣1287␣␣␣␣715␣␣␣␣286␣␣␣␣78␣␣␣␣␣13␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣14␣␣␣␣␣91␣␣␣␣␣364␣␣␣␣1001␣␣␣␣2002␣␣␣␣3003␣␣␣␣3432␣␣␣␣3003␣␣␣␣2002␣␣␣␣1001␣␣␣␣364␣␣␣␣91␣␣␣␣␣14␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣15␣␣␣␣␣105␣␣␣␣455␣␣␣␣1365␣␣␣␣3003␣␣␣␣5005␣␣␣␣6435␣␣␣␣6435␣␣␣␣5005␣␣␣␣3003␣␣␣␣1365␣␣␣␣455␣␣␣␣105␣␣␣␣15␣␣␣␣␣1␣␣
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣1␣␣␣␣␣16␣␣␣␣␣120␣␣␣␣560␣␣␣␣1820␣␣␣␣4368␣␣␣␣8008␣␣␣␣11440␣␣␣␣12870␣␣␣␣11440␣␣␣␣8008␣␣␣␣4368␣␣␣␣1820␣␣␣␣560␣␣␣␣120␣␣␣␣16␣␣␣␣␣1␣␣
```

While we have shown some trailing spaces in each row, which belong to the rightmost entry, in some of the above sample runs, omitting them is also allowed in our grading.

Here, we use the '␣' symbol to denote a single space character for clarity. But the real output consists of numbers and space characters only, like:

```
Enter num: 10↵
                    1
                  1   1
                1   2   1
              1   3   3   1
            1   4   6   4   1
          1   5  10  10   5   1
        1   6  15  20  15   6   1
      1   7  21  35  35  21   7   1
    1   8  28  56  70  56  28   8   1
  1   9  36  84  126 126 84  36   9   1
 1  10  45  120 210 252 210 120 45  10   1
```

Question 3 (20%)

An *esthetic number* is a positive integer where the absolute difference between every pair of its adjacent digits equals 1. See some examples below.

For base 10 (decimal),

- 23 is esthetic. Digits 2 and 3 differ by 1.
- 3454 is esthetic. Each pair of adjacent digits differ by 1.
- 441 is not esthetic. Leftmost digit and middle digit are both 4 (differ by 0).

For base 4, however,

- 441 is esthetic in base 4, as $441_{10} = 12321_4$. All adjacent digits differ by 1.
- 25 is esthetic in base 4, as $25_{10} = 121_4$. All adjacent digits differ by 1.

In fact, 25 is esthetic also in base 7 because $25_{10} = 34_7$. Digits 3 and 4 differ by 1.

Write a script called `q3.py` that defines a function `esthetic(n: int) -> list[int] | str`. This function should accept a positive integer n and return a list of bases (as integers from 2 to 10, inclusive) in which n is considered esthetic. If n is not esthetic in any of the bases (from 2 to 10), the function should return the string "`non-esthetic`". In the script (outside the function), prompt the user to enter a positive integer. Call the function with the provided input and print the return value.

Note: For bases under consideration where n is less than or equal to the base, it consists of a single digit only. For simplicity, consider n to be esthetic in that base.

Sample Runs

```
Enter num: 2↵
[2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
Enter num: 67673↵
non-esthetic
```

```
Enter num: 23↵
[3, 5, 7, 10]
```

The expected output for the last sample run can be explained as follows:

- 23 in base 3 = 212
- 23 in base 5 = 43
- 23 in base 7 = 32
- 23 in base 10 = 23

Question 4 (20%)

Logistic regression is a supervised machine learning algorithm used for binary classification, predicting the probability of an outcome (0 or 1, yes/no) based on predictor variables. It fits data to an S-shaped *sigmoid function* to map predicted values between 0 and 1, usually applying a 0.5 threshold to classify results. The essential mathematical formulae and steps of the algorithm are presented as follows.

The *sigmoid function* (a.k.a. *logistic function*) is defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \qquad \text{... (1)}$$

where z is a linear combination of inputs:

$$z = \mathbf{w} \cdot \mathbf{x} + b = \sum_{j=1}^{n} w_j x_j + b \qquad \text{... (2)}$$

$\mathbf{x} = [x_1, x_2, \dots, x_n]^T$ is the *feature vector*, a vector representing n input features or predictors for a single observation (data point). Each x_j is the value of the j-th feature for the specific observation. It contains all the necessary information for making a prediction regarding the class label for that sample.

$\mathbf{w} = [w_1, w_2, \dots, w_n]^T$ is the *weight vector*, a vector of n weights assigned to each feature in the model.

Given an input vector $\mathbf{x}$ and a weight vector $\mathbf{w}$, the probability that the output or class $Y = 1$ is:

$$P(Y = 1|\mathbf{x}) = \sigma(z) = \frac{1}{1 + e^{-(\mathbf{w}\cdot\mathbf{x}+b)}} \qquad \text{... (3)}$$

The probability for the class $Y = 0$ is simply:

$$P(Y = 0|\mathbf{x}) = 1 - P(Y = 1|\mathbf{x}) = 1 - \sigma(z) \qquad \text{... (4)}$$

Once the probability is calculated, a threshold (usually 0.5) is applied to make a final class prediction:
- If $P(Y = 1|\mathbf{x}) = \sigma(z) \geq 0.5$, predict Y as the class label 1.
- If $P(Y = 1|\mathbf{x}) = \sigma(z) < 0.5$, predict Y as the class label 0.

The cost function used to train the model is the *binary cross-entropy loss*, defined as

$$J(\mathbf{w}, b) = -\frac{1}{m}\sum_{i=1}^{m}\left[Y_i \ln\big(P(Y = 1|\mathbf{x}_i)\big) + (1 - Y_i)\ln\big(P(Y = 0|\mathbf{x}_i)\big)\right] \quad \text{... (5)}$$

where
- m is the number of training samples.
- $\mathbf{x}_i$ is the input vector of the i-th sample.
- Y_i is the actual binary label (0 or 1) for the i-th sample.

To optimize the weights and bias used in the prediction model, we use *gradient descent*:

$$w_j \leftarrow w_j - \alpha\frac{\partial J}{\partial w_j} \quad \text{... (6)}$$

$$b \leftarrow b - \alpha\frac{\partial J}{\partial b} \quad \text{... (7)}$$

where α is the *learning rate*.

The gradients for weights and bias can be calculated as follows (note: for simplicity, the derivation of these two formulae is skipped):

$$\frac{\partial J}{\partial w_j} = \frac{1}{m}\sum_{i=1}^{m}(P(Y = 1|\mathbf{x}_i) - Y_i)x_{ij} = \frac{1}{m}\sum_{i=1}^{m}(\sigma(z_i) - Y_i)x_{ij} \quad \text{... (8)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m}\sum_{i=1}^{m}(P(Y = 1|\mathbf{x}_i) - Y_i) \quad = \frac{1}{m}\sum_{i=1}^{m}(\sigma(z_i) - Y_i) \quad \text{... (9)}$$

where x_{ij} refers to the j-th feature (predictor) of the i-th sample.

The algorithm steps for implementing logistic regression are summarized as follows:

1. **Input**: Training data $(\mathbf{X}, \mathbf{Y})$ where $\mathbf{X}$ is the feature matrix (m samples by n features) and $\mathbf{Y}$ is the label vector (containing 0's or 1's).
2. **Initialization**:
   - Initialize the weights in $\mathbf{w}$ as zeros.
   - Initialize the bias b as zero.
3. **Training**:
   - For a fixed number of epochs (or until convergence):
     - Compute the predicted probabilities using the logistic function, i.e., formula (1) – (3).
     - Calculate the cost using the binary cross-entropy loss function, i.e., formula (5).
     - Update weights and bias using the gradients calculated, i.e., formulae (6) - (9).
4. **Output**: The learned weights $\mathbf{w}$ and bias b.
5. **Prediction**:
   - For a new sample $\mathbf{x}$, compute the predicted probability and classify based on a threshold (0.5).

Write a script `q4.py` that implements the following functions. A starter code file `q4_starter.py` has been provided. For better code readability, the starter code has defined a type alias called `Vector`, which represents a list of integer or floating-point values, and the `sigmoid()` function that computes the sigmoid of an input value z.

(a) `binary_cross_entropy(Y: Vector, Y_pred: Vector) -> float`:

This function accepts two lists:
- Y: the list of m true labels (binary values: 1 or 0).
- `Y_pred`: the list of m predicted probabilities (float values between 0 and 1).

The function should calculate and return the binary cross-entropy loss using the formula (5) with Y_i being `Y[i]`, $P(Y = 1|\mathbf{x}_i)$ being `Y_pred[i]`, and $P(Y = 0|\mathbf{x}_i)$ being `1 - Y_pred[i]`.

Note: The `log()` function provided by the `math` module calculates the natural logarithm.

(b) `fit(X: list[Vector], Y: Vector, epochs: int, learning_rate: float, tolerance: float) -> tuple[Vector, float]`:

This function trains a logistic regression model in an iterative manner. It accepts:
- X: A list of m feature vectors, where each feature vector is of length n (list of lists).
- Y: A corresponding list of m true labels.
- `epochs`: An integer specifying the maximum number of training epochs.
- `learning_rate`: A floating-point value for the learning rate used in updating weights.
- `tolerance`: A floating-point value indicating when to stop training based on the cost.

The function should perform the following:
- Initialize the weight vector $\mathbf{w}$ (a list of n `float` values) to zeros and bias b to zero.
- For each epoch, derive a list `Y_pred` of m predicted probabilities using the current weights and bias using the formulae (1) - (3) and the given `sigmoid()` function.
- Update the n weights and bias using the m pairs of true labels (i.e., `Y[i]`) and predicted probabilities (i.e., `Y_pred[i]`) via gradient descent, i.e., formulae (6) - (9).
- Calculate the cost using the `binary_cross_entropy()` function with Y and `Y_pred`.
- Break from the training loop if the calculated cost is less than the given tolerance.
- Print the message "`Training stopped at epoch {`*epoch*`}, Cost: {`*cost*`}`". The cost is printed with 5 decimal places.

It returns a tuple containing:
- The final weights $\mathbf{w}$ (list of `float` values).
- The final bias b (`float` value).

(c) `predict(X: list[Vector], w: Vector, b: float) -> Vector`:

This function predicts class labels (0 or 1) for the given feature vectors. It accepts:
- X: A list of m feature vectors (list of lists).
- w: A list of weights (list of `float` values).
- b: A `float` value representing the bias.

The function should compute the predicted probability for each input feature vector by applying the `sigmoid()` function on the input linear combination:

$$z = \sum_{j=1}^{n} w[j] \cdot X[i][j] + b$$

If the probability is greater than or equal to 0.5, classify it as 1; otherwise, classify it as 0.

The function returns a list of predicted class labels (0 or 1) for all inputs.

Sample Client

```
if __name__ == '__main__':
    # Example dataset
    X = [[1, 2], [2, 3], [3, 3], [5, 6], [8, 9], [1, 1]]
    Y = [0, 0, 0, 1, 1, 0]  # Corresponding labels

    # Create a logistic regression model
    W, b = fit(X, Y, learning_rate=0.05, epochs=5000, tolerance=0.05)

    # Predictions
    predictions = predict(X, W, b)

    print("Weights: [" + ", ".join(f"{w:.5f}" for w in W) + "]")
    print(f"Bias: {b:.5f}")
    print("Predictions:", predictions)

    # Another example dataset
    X = [[1, 2, 1], [8, 7, 9], [5, 1, 2], [2, 4, 3], [9, 9, 7], [1, 1, 0]]
    Y = [0, 1, 0, 0, 1, 0]  # Corresponding labels

    # Create another logistic regression model
    W, b = fit(X, Y, learning_rate=0.05, epochs=3000, tolerance=0.03)

    # Predictions
    predictions = predict(X, W, b)

    print("Weights: [" + ", ".join(f"{w:.5f}" for w in W) + "]")
    print(f"Bias: {b:.5f}")
    print("Predictions:", predictions)
```

Sample Run

```
Training stopped at epoch 3176, Cost: 0.04999
Weights: [0.75884, 0.92206]
Bias: -7.06916
Predictions: [0, 0, 0, 1, 1, 0]
Training stopped at epoch 2684, Cost: 0.03000
Weights: [0.22595, 0.13746, 0.88914]
Bias: -6.15401
Predictions: [0, 1, 0, 0, 1, 0]
```

Question 5 (20%)

K-medoids clustering is an unsupervised machine learning algorithm used to partition a dataset into k distinct, non-overlapping clusters. While it is fundamentally similar to the popular *K-means* clustering, it uses actual data points as cluster centers (*medoids*) instead of the average (*mean*) of the points in the cluster. This makes K-medoids more robust to outliers. The steps of the clustering algorithm to be implemented in this question are outlined as follows.

Assumptions:
The dataset consists of distinct data points, each of which is a vector in an n-dimensional space.

1. **Initialization:**
- Suppose that the number of clusters to form from the dataset is k. Randomly select k distinct data points as initial medoids. Let's call this list of length k the `medoids` list.

2. **Assignment Step:**
- For each data point, calculate its Manhattan distance from each medoid in the `medoids` list.

  For two vectors $\boldsymbol{p} = (p_1, p_2, \dots, p_n)$ and $\boldsymbol{q} = (q_1, q_2, \dots, q_n)$ in n-dimensional space, the Manhattan distance between them is given by:

$$d(\boldsymbol{p}, \boldsymbol{q}) = \sum_{i=1}^{n} |p_i - q_i|$$

- Assign each data point to the cluster associated with the nearest medoid. By this step, each cluster should contain at least one point (the medoid).

3. **Update Step:**
- For each cluster, select the point within the cluster that minimizes the sum of distances to all other points in that cluster to be the new medoid. That is, for a cluster C, its new medoid M_C is derived by:

$$M_C = \underset{\boldsymbol{c} \in C}{\operatorname{argmin}} \sum_{\boldsymbol{p} \in C} d(\boldsymbol{c}, \boldsymbol{p})$$

  where
  - c is a candidate for the new medoid.
  - $d(\boldsymbol{c}, \boldsymbol{p})$ is the distance between points $\boldsymbol{c}$ and $\boldsymbol{p}$.

- This step replaces the list of old medoids with a new list.

4. **Convergence Check:**
- Repeat the assignment and update steps until medoids do not change or a maximum number of iterations is reached.

5. **Output:**
- Return the final medoids and clusters.

Your task in this question is to implement the above algorithm using Python lists to store the data points, medoids, and clusters. A starter code file `q5_starter.py` has been provided. Rename it and add your code in the file. The starter code has defined a type alias called `Vector` for a list of integer or floating-point values to represent a n-dimensional point.

(a) Write a function called `manhattan_distance(point1: Vector, point2: Vector) -> float | int`: Calculate the Manhattan distance between two points.

(b) Write a function called `k_medoids(data: list[Vector], k: int = 3, max_iterations: int = 100, seed: int = 42) -> tuple[list[Vector], list[list[Vector]]]` to implement the *k*-medoids clustering algorithm presented above from scratch.

Note that `data` is the list of data points. k is the number of clusters to form. This function begins with Step 1, creating a list of initial medoids which are set to k randomly picked data points in `data`. Since different ways of random sampling and using a different value for the random seed may result in different results, the starter code has done Step 1 for you. It uses 42 as the default seed for the random number generator and the `sample()` function in the `random` module to randomly pick k data points as the initial medoids. ***DO NOT modify this part of given code*** *(except local variable names if you like to)* ***or else you may fail our test cases that assumed this way of initialization.***

The function iterates at most `max_iterations` rounds of Steps 2 to 4 and returns a 2-tuple that contains the list of k *final medoids* and the *clusters*. The loop breaks earlier if Step 4 above finds no difference between the old list and new list of medoids after the update step.

To make the code more modular, the assignment step (Step 2) and update step (Step 3) are to be factored out into two helper functions described in part (c) and (d) below. In other words, your `k_medoids` function body should call them to carry out Step 2 and 3 respectively.

(c) Write a helper function called `assign_clusters(data: list[Vector], medoids: list[Vector]) -> list[list[Vector]]`, which takes the list of data points and the list of medoids as input, and returns the list of clusters formed.

First of all, create a nested list containing `len(medoids)` empty lists. This nested list serves as the list of clusters to be returned by this function.

For each point in `data`, select the medoid having the shortest Manhattan distance from the point and append the point to the cluster associated with that medoid. The function returns the nested list of clusters to the caller for subsequent processing.

(d) Write a helper function called `update_medoids(clusters: list[list[Vector]]) -> list[Vector]`, which takes the list of clusters and returns a new list of medoids to the caller.

Despite the name, this function does not update the existing list of medoids; instead, it creates a new one. The new list is created as follows: for each non-empty cluster in `clusters`, each point in the cluster considered a candidate, and the total distance from all points in the cluster to each candidate is calculated. The candidate with the minimum total distance is then selected as the new medoid.

Sample Client

A sample client code has been given in the starter code as follows. It tests the implemented algorithm with two sample datasets. Your client code in this question will NOT be graded, so you can feel free to modify it to add more test cases if you like to.

```python
# Sample client
if __name__ == '__main__':
    # Sample data 1
    data = [
        (1, 2), (1, 4),                  # Cluster 1
        (10, 2), (10, 4),                # Cluster 2
        (2, 3), (3, 4),                  # Cluster 1
        (12, 3), (11, 2),                # Cluster 2
        (5, 8), (6, 9), (7, 7), (6, 8),  # Cluster 3
        (90, 50), (68, 47)               # Cluster 4 (Outliners)
    ]
    medoids, clusters = k_medoids(data, k=4, seed=99)
    print("Sample Run 1")
    print("Final Medoids:", medoids)
    for index, cluster in enumerate(clusters):
        print(f"Cluster {index + 1}: {cluster}")

    # Sample data 2
    data = [
        [1.0, 2.0, 3.0],     # Cluster 1
        [1.5, 1.8, 2.5],     # Cluster 1
        [5.5, 6.5, 7.0],     # Cluster 2
        [1.0, 0.5, 1.5],     # Cluster 1
        [6.0, 7.0, 6.5],     # Cluster 2
        [99.9, 89.0, 79.5],  # Cluster 3 (Outliner)
        [4.5, 5.0, 5.5],     # Cluster 2
        [1.8, 2.2, 2.0],     # Cluster 1

    ]
    medoids, clusters = k_medoids(data, k=3, seed=13)
    print("Sample Run 2")
    print("Final Medoids:", medoids)
    for index, cluster in enumerate(clusters):
        print(f"Cluster {index + 1}: {cluster}")
```

Sample Runs

```
Sample Run 1
Final Medoids: [(10, 2), (90, 50), (1, 4), (6, 8)]
Cluster 1: [(10, 2), (10, 4), (12, 3), (11, 2)]
Cluster 2: [(90, 50), (68, 47)]
Cluster 3: [(1, 2), (1, 4), (2, 3), (3, 4)]
Cluster 4: [(5, 8), (6, 9), (7, 7), (6, 8)]
Sample Run 2
Final Medoids: [[5.5, 6.5, 7.0], [1.5, 1.8, 2.5], [99.9, 89.0, 79.5]]
Cluster 1: [[5.5, 6.5, 7.0], [6.0, 7.0, 6.5], [4.5, 5.0, 5.5]]
Cluster 2: [[1.0, 2.0, 3.0], [1.5, 1.8, 2.5], [1.0, 0.5, 1.5], [1.8, 2.2, 2.0]]
Cluster 3: [[99.9, 89.0, 79.5]]
```

Note: It is normal that the cluster ids in the output `labels` list do not match with the ones written in the comments around the `data` list in the sample client code. As long as the data points marked with the same comment, say "`# Cluster 1`", in the client code are given the same cluster id, the result is fine. The clustering result may also be highly dependent on the seed value, which affects random sampling of the initial medoids.